📖 Documentación Técnica — Zzaly E-commerce

> **Versión:** 1.0.0
> **Última actualización:** 27 de mayo de 2026
> **Plataforma:** E-commerce de moda femenina — Mercado Guatemala (GTQ)

📑 Tabla de Contenidos

1. Descripción General del Proyecto
2. Arquitectura del Sistema
3. Estructura del Proyecto
4. Instalación y Configuración
5. Variables de Entorno
6. Stack de Dependencias
7. Backend — Servidor Express
    - 7.1 Punto de Entrada y Middleware
    - 7.2 Referencia Completa de la API (23 endpoints)
    - 7.3 Autenticación y Autorización
8. Base de Datos — PostgreSQL
    - 8.1 Diagrama Entidad-Relación
    - 8.2 Tipos ENUM
    - 8.3 Esquema de Tablas
    - 8.4 Procedimiento Almacenado: process_checkout()
    - 8.5 Triggers
    - 8.6 Datos Semilla
9. Frontend — React SPA
    - 9.1 Enrutamiento
    - 9.2 Gestión de Estado
    - 9.3 Sistema de Diseño
    - 9.4 Componentes Principales
10. Seguridad
11. Despliegue a Producción
12. Guía de Contribución
13. Solución de Problemas

---

1 · Descripción General del Proyecto

Zzaly es una plataforma de comercio electrónico especializada en **moda femenina**, diseñada específicamente para el mercado de **Guatemala**. Todas las transacciones se realizan en **Quetzales (GTQ)** y la interfaz de usuario está completamente en **español**.

Característica	Detalle
Nombre	Zzaly
Tipo	E-commerce B2C
Nicho	Moda femenina
Mercado	Guatemala
Moneda	GTQ (Quetzales guatemaltecos)
Idioma UI	Español
Arquitectura	Cliente-Servidor (SPA + REST API)

Funcionalidades Principales

- 🛍️ Catálogo de productos con filtros avanzados (categoría, talla, color, precio)
- 🛒 Carrito de compras persistente en `localStorage`
- 🎟️ Sistema de códigos promocionales con validaciones
- 📦 Proceso de checkout con transacciones ACID
- 🔐 Panel de administración protegido con JWT
- 📊 Dashboard administrativo con métricas en tiempo real
- 📱 Diseño responsive con enfoque mobile-first

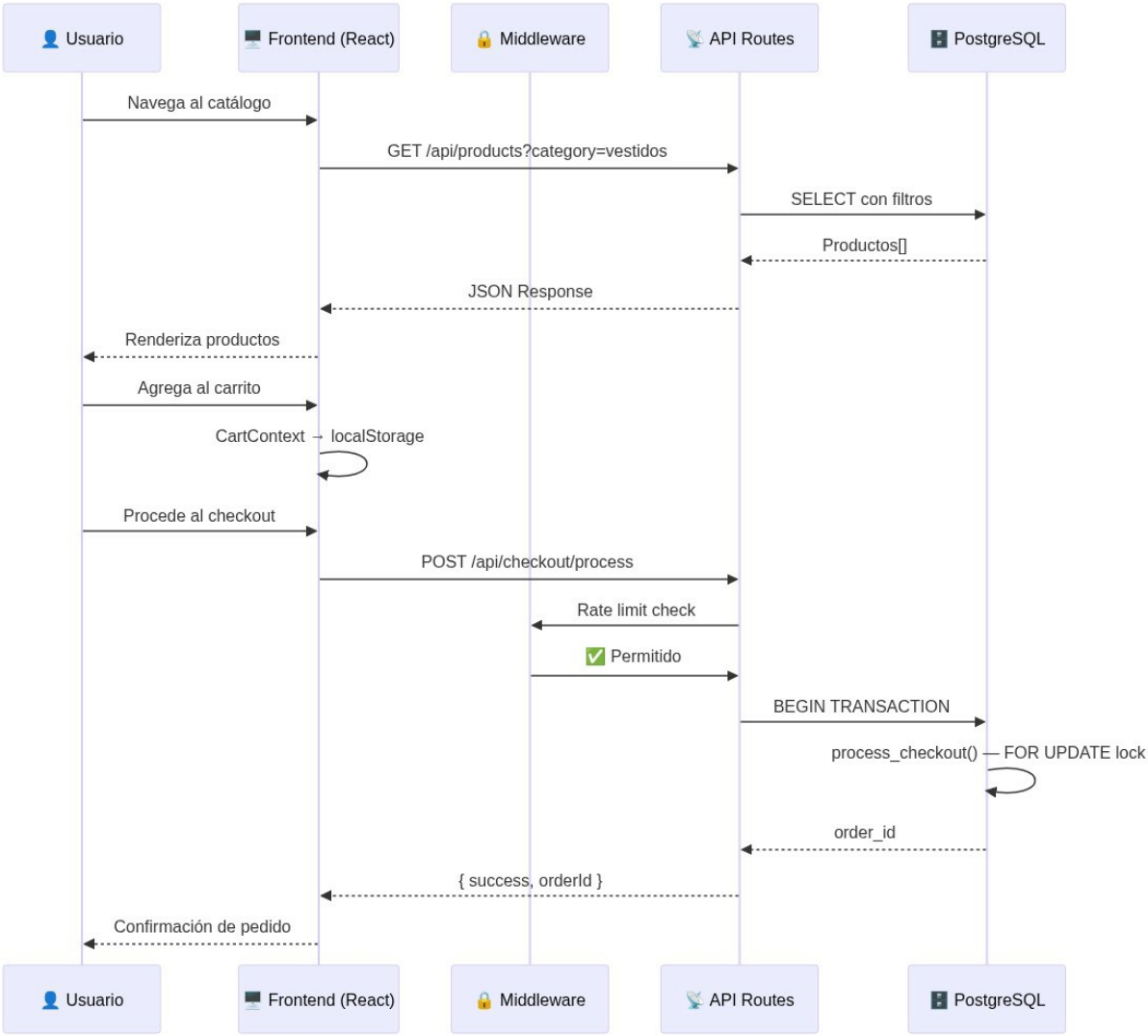
---

2 · Arquitectura del Sistema

Diagrama de Arquitectura General



Flujo de Datos Principal



---

3 · Estructura del Proyecto

```
Ecommerce/
│
├── client/                          # Frontend — React SPA
│   ├── public/                      # Recursos estáticos (imágenes)
│   ├── src/
│   │   ├── components/              # Componentes reutilizables
│   │   │   ├── CatalogFilters.jsx   # Filtros del catálogo
│   │   │   ├── Footer.jsx           # Pie de página
│   │   │   ├── HeroCarousel.jsx     # Carrusel principal (Swiper)
│   │   │   ├── Navbar.jsx           # Barra de navegación
│   │   │   ├── ProductCard.jsx      # Tarjeta de producto
│   │   │   └── ProductDetail.jsx    # Modal detalle de producto
│   │   │
│   │   ├── context/
│   │   │   └── CartContext.jsx      # Estado global del carrito
│   │   │
│   │   ├── hooks/
│   │   │   └── useScrollReveal.js   # Hook de animación scroll
│   │   │
│   │   ├── pages/
│   │   │   ├── Admin/               # Panel administrativo
│   │   │   │   ├── AdminDashboard.jsx  # Dashboard con métricas
│   │   │   │   ├── AdminLayout.jsx     # Layout del panel admin
│   │   │   │   ├── AdminLogin.jsx      # Inicio de sesión admin
│   │   │   │   ├── AdminOrders.jsx     # Gestión de pedidos
│   │   │   │   ├── AdminProducts.jsx   # CRUD de productos
│   │   │   │   └── AdminPromos.jsx     # Gestión de promos
│   │   │   │
│   │   │   ├── Cart.jsx             # Página del carrito
│   │   │   ├── Catalog.jsx          # Catálogo con filtros
│   │   │   ├── Home.jsx             # Página de inicio
│   │   │   └── Nosotros.jsx         # Sobre nosotros
│   │   │
│   │   ├── App.jsx                  # Componente raíz + rutas
│   │   ├── App.css                  # Estilos de la app
│   │   ├── index.css                # Sistema de diseño (3000+ líneas)
│   │   └── main.jsx                 # Punto de entrada React
│   │
│   ├── .env                         # Variables de entorno del cliente
│   ├── index.html                   # HTML base (SPA)
│   ├── package.json                 # Dependencias del frontend
│   └── vite.config.js               # Configuración de Vite
│
├── server/                          # Backend — Express.js
│   ├── db/
│   │   ├── pool.js                  # Conexión a PostgreSQL
│   │   ├── schema.sql               # Esquema completo de la BD
│   │   ├── seed.js                  # Datos semilla (productos)
│   │   ├── seed_admin.js            # Datos semilla (admin)
│   │   └── migration_admins.sql     # Migración tabla admins
│   │
│   ├── routes/
│   │   ├── admin.js                 # CRUD admin (protegido)
│   │   ├── auth.js                  # Login/verificación JWT
│   │   ├── categories.js            # Categorías públicas
│   │   ├── checkout.js              # Procesamiento de pedidos
│   │   ├── products.js              # Productos públicos
│   │   └── promos.js                # Validación de promos
│   │
│   ├── .env                         # Variables de entorno
│   ├── .env.example                 # Plantilla de variables
│   ├── index.js                     # Punto de entrada Express
│   └── package.json                 # Dependencias del backend
│
└── render.yaml                      # Configuración de despliegue Render
```

---

4 · Instalación y Configuración

Requisitos Previos

Herramienta	Versión Mínima	Propósito
Node.js	18.x o superior	Entorno de ejecución
npm	9.x o superior	Gestor de paquetes
PostgreSQL	14.x o superior	Base de datos (local o Supabase)
Git	2.x	Control de versiones

Paso 1 — Clonar el repositorio

```
git clone <URL_DEL_REPOSITORIO>
cd Ecommerce
```

Paso 2 — Configurar el Backend

```
# Instalar dependencias del servidor
cd server
npm install

# Crear archivo de variables de entorno
cp .env.example .env
```

> [!IMPORTANT]
> Edita el archivo `server/.env` con las credenciales de tu base de datos PostgreSQL antes de continuar. Consulta la sección Variables de Entorno para el detalle completo.

Paso 3 — Configurar la Base de Datos

```
# Opción A: Si tienes PostgreSQL local
psql -U postgres -c "CREATE DATABASE zzaly;"
psql -U postgres -d zzaly -f db/schema.sql

# Ejecutar datos semilla
node db/seed.js
node db/seed_admin.js
```

```
# Opción B: Con DATABASE_URL (Supabase u otro servicio)
# Asegúrate de que DATABASE_URL esté configurado en .env
# Luego ejecuta el schema manualmente desde el cliente de Supabase
# o via psql:
psql "$DATABASE_URL" -f db/schema.sql
node db/seed.js
node db/seed_admin.js
```

Paso 4 — Configurar el Frontend

```
# Desde la raíz del proyecto
cd client
npm install

# Crear archivo de variables de entorno
echo "VITE_API_URL=http://localhost:5000/api" > .env
```

Paso 5 — Ejecutar en Desarrollo

```
# Terminal 1 — Servidor (desde /server)
cd server
npm start
# ☑ Servidor corriendo en http://localhost:5000

# Terminal 2 — Cliente (desde /client)
cd client
npm run dev
# ☑ Frontend corriendo en http://localhost:5173
```

> [!TIP]
> Para desarrollo, asegúrate de que `VITE_API_URL` apunte a `http://localhost:5000/api` en el archivo `client/.env`.

Verificación Rápida

```
# Verificar que el backend responde
curl http://localhost:5000/api/health
# Respuesta esperada: { "status": "ok", "timestamp": "..." }

# Verificar que los productos se cargaron
curl http://localhost:5000/api/products
# Respuesta esperada: Array de productos
```

---

5 · Variables de Entorno

Servidor (`server/.env`)

Variable	Valor por Defecto	Requerida	Descripción
`DATABASE_URL`	*(construida desde partes)*	Recomendada	Cadena de conexión completa a PostgreSQL. Si se proporciona, tiene prioridad sobre las variables `DB_*` individuales.
`DB_HOST`	`localhost`	Si no hay `DATABASE_URL`	Host del servidor de base de datos.
`DB_PORT`	`5432`	Si no hay `DATABASE_URL`	Puerto de PostgreSQL.
`DB_USER`	`postgres`	Si no hay `DATABASE_URL`	Usuario de la base de datos.
`DB_PASSWORD`	`""`	Si no hay `DATABASE_URL`	Contraseña del usuario de la BD.
`DB_NAME`	`zzaly`	Si no hay `DATABASE_URL`	Nombre de la base de datos.
`PORT`	`5000`	No	Puerto en el que escucha Express.

`JWT_SECRET`	`zzaly-super-secret-jwt-key-2026`	**Sí (producción)**	Clave secreta para firmar tokens JWT. **Cambiar obligatoriamente en producción.**
`FRONTEND_URL`	*(ninguno)*	No	Origen permitido para CORS. Útil para producción.

> [!CAUTION]
> **Nunca** utilices el `JWT_SECRET` por defecto en producción. Genera una clave aleatoria segura de al menos 64 caracteres.

Ejemplo de `server/.env` para desarrollo local:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=mi_password_seguro
DB_NAME=zzaly
PORT=5000
JWT_SECRET=clave-desarrollo-local-no-usar-en-prod
```

Ejemplo de `server/.env` para producción (Supabase):

```
DATABASE_URL=postgresql://postgres.xxxx:password@aws-0-us-east-1.pooler.supabase.com:6543/postgres
PORT=5000
JWT_SECRET=una-clave-muy-larga-y-aleatoria-generada-de-forma-segura-64chars
FRONTEND_URL=https://mi-tienda-zzaly.netlify.app
```

Cliente (`client/.env`)

Variable	Valor	Descripción
`VITE_API_URL`	`https://ecommerce-backend-mi96.onrender.com/api`	URL base de la API del backend. Cambia a `http://localhost:5000/api` para desarrollo local.

> [!NOTE]
> Las variables del cliente con prefijo `VITE_` son expuestas al navegador por Vite. **Nunca** coloques secretos o credenciales sensibles en el archivo `.env` del cliente.

---

6 · Stack de Dependencias

Servidor (Backend)

Paquete	Versión	Propósito
`express`	^4.21.2	Framework web principal para la API REST
`cors`	^2.8.5	Manejo de Cross-Origin Resource Sharing
`helmet`	^8.1.0	Cabeceras de seguridad HTTP
`express-rate-limit`	^8.2.1	Limitación de tasa de peticiones
`dotenv`	^16.4.7	Carga de variables de entorno desde `.env`
`pg`	^8.13.1	Cliente PostgreSQL para Node.js
`pg-connection-string`	^2.11.0	Parser de cadenas de conexión de PostgreSQL
`bcryptjs`	^3.0.3	Hash de contraseñas (bcrypt en JS puro)
`jsonwebtoken`	^9.0.3	Generación y verificación de tokens JWT

Cliente (Frontend)

Paquete	Versión	Propósito
`react`	^19.2.0	Biblioteca de interfaz de usuario
`react-dom`	^19.2.0	Renderizador DOM de React
`react-router-dom`	^7.13.1	Enrutamiento del lado del cliente (SPA)
`swiper`	^12.1.2	Componente de carrusel/slider
`vite`	7.3.1	Herramienta de build y servidor de desarrollo
`@vitejs/plugin-react`	*(devDep)*	Plugin de Vite para soporte React

---

7 · Backend — Servidor Express

7.1 Punto de Entrada y Middleware

El servidor se inicializa en `server/index.js` con la siguiente cadena de middleware aplicada en orden:

Diagrama

#	Middleware	Configuración	Propósito
1	**Helmet**	`crossOriginResourcePolicy: 'cross-origin'`	Establece cabeceras de seguridad HTTP estándar. Permite carga de recursos cross-origin para imágenes de productos.
2	**CORS**	Orígenes dinámicos	Permite peticiones desde `localhost:5173`, `localhost:3000`, `FRONTEND_URL`, y `*.netlify.app`.
3	**JSON Body Parser**	`express.json({ limit: '10mb' })`	Parsea cuerpos de petición JSON con un límite de 10 MB para soportar datos de productos con imágenes.
4	**Rate Limiter General**	`200 peticiones / 15 minutos`	Se aplica a todas las rutas bajo `/api/*`. Protege contra abuso de la API.
5	**Rate Limiter Login**	`10 peticiones / 15 minutos`	Específico para `/api/auth/login`. Previene ataques de fuerza bruta.

Orígenes CORS permitidos:

```
// Configuración dinámica de CORS
const allowedOrigins = [
  'http://localhost:5173',     // Vite dev server
  'http://localhost:3000',     // Alternativo
  process.env.FRONTEND_URL,  // Producción (configurable)
  /\.netlify\.app$/            // Cualquier subdominio de Netlify
];
```

7.2 Referencia Completa de la API (23 endpoints)

> [!NOTE]
> Todas las rutas devuelven respuestas en formato **JSON**. Los códigos de error siguen el estándar HTTP. Las rutas de administración requieren un token JWT válido en la cabecera `Authorization`.

Mapa de Rutas

Diagrama

📌 Raíz y Estado del Servidor

`GET /`

Mensaje de bienvenida y enlace a documentación.

```
curl http://localhost:5000/
```

```
{
  "message": "Zzaly E-Commerce API",
  "docs": "/api/health"
}
```

`GET /api/health`

Verificación del estado del servidor.

```
curl http://localhost:5000/api/health
```

```
{
  "status": "ok",
  "timestamp": "2026-05-27T01:13:00.000Z"
}
```

🛍️ Productos (Público)

`GET /api/products`

Lista productos con filtros opcionales. Retorna datos enriquecidos con imágenes, variantes, tallas y colores disponibles.

Parámetros de Query:

Parámetro	Tipo	Descripción	Ejemplo
`category`	`string`	Slug de la categoría	`vestidos`
`size`	`string`	Talla del producto	`M`
`color`	`string`	Color del producto	`Negro`
`min_price`	`number`	Precio mínimo (GTQ)	`100`
`max_price`	`number`	Precio máximo (GTQ)	`500`
`featured`	`string`	Solo productos destacados	`true`
`search`	`string`	Búsqueda por nombre (ILIKE)	`vestido floral`

Ejemplo — Todos los productos:

```
curl "http://localhost:5000/api/products"
```

Ejemplo — Filtrado combinado:

```
curl "http://localhost:5000/api/products?category=vestidos&size=M&min_price=100&max_price=500&featured=true"
```

Respuesta:

```
[
  {
    "id": 1,
    "name": "Vestido Floral de Verano",
    "slug": "vestido-floral-de-verano",
    "description": "Un hermoso vestido con estampado floral...",
    "price": "249.99",
    "category_id": 1,
    "category_name": "Vestidos",
    "category_slug": "vestidos",
    "is_featured": true,
    "is_active": true,
    "images": [
      {
        "id": 1,
        "url": "https://ejemplo.com/imagen1.jpg",
        "alt_text": "Vestido floral vista frontal",
        "sort_order": 0
      }
    ],
    "variants": [
      {
        "id": 1,
        "size": "S",
        "color": "Rosa",
        "color_hex": "#FFB6C1",
        "stock": 10,
        "sku": "VFV-S-ROSA"
      }
    ],
    "sizes": ["XS", "S", "M", "L", "XL"],
    "colors": ["Rosa", "Blanco", "Negro"]
  }
]
```

`GET /api/products/:id`

Obtiene un producto individual con toda su información.

```
curl http://localhost:5000/api/products/1
```

Respuesta exitosa: Objeto de producto completo (mismo formato que el array anterior).

Error 404:


```
{
  "error": "Producto no encontrado"
}
```

🗁 Categorías (Público)

`GET /api/categories`

Lista todas las categorías ordenadas alfabéticamente por nombre.

```
curl http://localhost:5000/api/categories
```


```
[
  { "id": 1, "name": "Vestidos", "slug": "vestidos" },
  { "id": 2, "name": "Blusas", "slug": "blusas" },
  { "id": 3, "name": "Faldas", "slug": "faldas" },
  { "id": 4, "name": "Pantalones", "slug": "pantalones" },
  { "id": 5, "name": "Accesorios", "slug": "accesorios" },
  { "id": 6, "name": "Calzado", "slug": "calzado" }
]
```

🎟 Códigos Promocionales (Público)

`POST /api/promos/validate`

Valida un código promocional antes de aplicarlo al carrito.

```
curl -X POST http://localhost:5000/api/promos/validate \
  -H "Content-Type: application/json" \
  -d '{ "code": "WELCOME10", "subtotal": 300 }'
```

Cuerpo de la petición:

Campo	Tipo	Requerido	Descripción
`code`	`string`	☑	Código promocional a validar
`subtotal`	`number`	☑	Subtotal actual del carrito (GTQ)

Validaciones ejecutadas (en orden):

1. ☑ El código existe y está activo (`is_active = true`)
2. ☑ No ha expirado (`valid_until` es nulo o futuro)

3. ☑ No ha alcanzado el límite de usos (`used_count < max_uses`)
4. ☑ El subtotal cumple el mínimo requerido (`subtotal >= min_order_value`)

Respuesta exitosa:

```
{
  "id": 1,
  "code": "WELCOME10",
  "discount_type": "percentage",
  "discount_value": "10.00"
}
```

Errores posibles:

Código	Mensaje	Causa
400	`"El código es requerido"`	No se envió `code` en el body
404	`"Código promocional no válido o expirado"`	No existe, inactivo o expirado
400	`"Este código ha alcanzado su límite de usos"`	`used_count >= max_uses`
400	`"El pedido mínimo para este código es Q{valor}"`	`subtotal < min_order_value`

🛒 Checkout (Público)

`POST /api/checkout/process`

Procesa un pedido completo utilizando una **transacción ACID** con bloqueo a nivel de fila.

```
curl -X POST http://localhost:5000/api/checkout/process \
  -H "Content-Type: application/json" \
  -d '{
    "userId": null,
    "subtotal": 499.98,
    "discountAmount": 49.99,
    "total": 449.99,
    "promoCodeId": 1,
    "shippingAddress": {
      "street": "6ta Avenida 12-34",
      "city": "Guatemala",
      "state": "Guatemala",
      "zip": "01001"
    },
    "contactEmail": "cliente@ejemplo.com",
    "contactPhone": "+502 5555-1234",
    "items": [
      {
        "variantId": 5,
        "productName": "Vestido Floral de Verano",
        "size": "M",
        "color": "Rosa",
        "quantity": 2,
        "unitPrice": 249.99,
        "subtotal": 499.98
      }
    ]
  }'
```

Cuerpo de la petición:

Campo	Tipo	Requerido	Descripción	
`userId`	`UUID \`	null`	No	ID del usuario (null para invitados)
`subtotal`	`number`	☑	Subtotal antes de descuentos	
`discountAmount`	`number`	No	Monto del descuento aplicado	
`total`	`number`	☑	Total final a cobrar	
`promoCodeId`	`integer \`	null`	No	ID del código promo utilizado
`shippingAddress`	`object (JSONB)`	☑	Dirección de envío	
`contactEmail`	`string`	☑	Email de contacto del comprador	
`contactPhone`	`string`	No	Teléfono de contacto	
`items`	`array`	☑	Array de artículos del pedido	

Estructura de cada `item`:

Campo	Tipo	Descripción
`variantId`	`integer`	ID de la variante del producto
`productName`	`string`	Nombre del producto (snapshot)
`size`	`string`	Talla seleccionada
`color`	`string`	Color seleccionado
`quantity`	`integer`	Cantidad solicitada
`unitPrice`	`number`	Precio unitario
`subtotal`	`number`	Subtotal de la línea

Flujo interno del proceso:

Diagrama

Respuesta exitosa (201):

```
{
  "success": true,
  "message": "Pedido procesado exitosamente",
  "orderId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "paymentId": "f9e8d7c6-b5a4-3210-fedc-ba0987654321",
  "totalToPay": 449.99
}
```

Errores posibles:

Código	Situación
400	Carrito vacío o datos obligatorios faltantes
409	Stock insuficiente para uno o más artículos
500	Error interno del servidor

🔑 Autenticación (Público)

`POST /api/auth/login`

Inicia sesión como administrador.

> [!WARNING]
> Este endpoint tiene un rate limit de **10 peticiones por 15 minutos** para prevenir ataques de fuerza bruta.

```
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{ "email": "admin@zzaly.com", "password": "Zzaly2026!Secure" }'
```

Cuerpo de la petición:

Campo	Tipo	Requerido	Descripción
`email`	`string`	☑	Email del administrador
`password`	`string`	☑	Contraseña en texto plano

Respuesta exitosa:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "admin": {
    "id": 1,
    "email": "admin@zzaly.com",
    "name": "Admin"
  }
}
```

Errores:

Código	Mensaje
400	Campos requeridos faltantes
401	Credenciales inválidas
500	Error interno

`POST /api/auth/verify`

Verifica la validez de un token JWT existente.

```
curl -X POST http://localhost:5000/api/auth/verify \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..."
```

Respuesta exitosa:

```
{
  "valid": true,
  "admin": {
    "id": 1,
    "email": "admin@zzaly.com",
    "name": "Admin"
  }
}
```

Error 401:

```
{
  "valid": false
}
```

🔒 Administración (Requiere JWT)

> [!IMPORTANT]
> **Todas** las rutas bajo `/api/admin/*` requieren autenticación. Incluye la cabecera `Authorization: Bearer <token>` en cada petición.

Middleware `isAdmin`:

1. Extrae el token de la cabecera `Authorization: Bearer <token>`
2. Verifica la firma JWT con `JWT_SECRET`
3. Decodifica el payload y asigna `req.admin = { id, email, name }`
4. Si el token es inválido o ausente → `401 No autorizado`

`GET /api/admin/dashboard`

Retorna métricas generales para el panel de administración.

```
curl http://localhost:5000/api/admin/dashboard \
  -H "Authorization: Bearer <TOKEN>"
```

```
{
  "totalProducts": 12,
  "totalOrders": 45,
  "totalRevenue": 15780.50,
  "lowStockAlerts": 3,
  "recentOrders": [
    {
      "id": "uuid-del-pedido",
      "status": "pending",
      "total": "449.99",
      "contact_email": "cliente@ejemplo.com",
      "created_at": "2026-05-27T01:00:00.000Z"
    }
  ]
}
```

Productos CRUD — Admin

Método	Ruta	Descripción
GET	`/api/admin/products`	Lista todos los productos con variantes, imágenes y categoría
GET	`/api/admin/products/:id`	Detalle de un producto
POST	`/api/admin/products`	Crear nuevo producto (transaccional: producto + variantes + imágenes)
PUT	`/api/admin/products/:id`	Actualizar producto (transaccional: reemplaza variantes e imágenes)
DELETE	`/api/admin/products/:id`	Eliminar producto (cascada)

Ejemplo — Crear producto:

```
curl -X POST http://localhost:5000/api/admin/products \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Blusa de Seda Premium",
    "slug": "blusa-de-seda-premium",
    "description": "Elegante blusa de seda natural con caída suave.",
    "price": 349.99,
    "category_id": 2,
    "is_featured": true,
    "is_active": true,
    "variants": [
      { "size": "S", "color": "Blanco", "color_hex": "#FFFFFF", "stock": 15, "sku": "BSP-S-BL" },
      { "size": "M", "color": "Blanco", "color_hex": "#FFFFFF", "stock": 20, "sku": "BSP-M-BL" },
      { "size": "L", "color": "Negro", "color_hex": "#000000", "stock": 10, "sku": "BSP-L-NG" }
    ],
    "images": [
      { "url": "https://ejemplo.com/blusa-1.jpg", "alt_text": "Blusa seda vista frontal", "sort_order": 0 },
      { "url": "https://ejemplo.com/blusa-2.jpg", "alt_text": "Blusa seda vista lateral", "sort_order": 1 }
    ]
  }'
```

Ejemplo — Actualizar producto:

```
curl -X PUT http://localhost:5000/api/admin/products/1 \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Vestido Floral de Verano — Edición Limitada",
    "price": 279.99,
    "variants": [ ... ],
    "images": [ ... ]
  }'
```

Ejemplo — Eliminar producto:

```
curl -X DELETE http://localhost:5000/api/admin/products/1 \
  -H "Authorization: Bearer <TOKEN>"
```

Pedidos — Admin

Método	Ruta	Descripción
GET	`/api/admin/orders`	Lista todos los pedidos con items agregados
GET	`/api/admin/orders/:id`	Detalle de un pedido con items
PUT	`/api/admin/orders/:id/status`	Actualizar estado del pedido

Estados válidos para pedidos:

Ejemplo — Actualizar estado:

```
curl -X PUT http://localhost:5000/api/admin/orders/uuid-del-pedido/status \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{ "status": "shipped" }'
```

```
{
  "id": "uuid-del-pedido",
  "status": "shipped",
  "updated_at": "2026-05-27T02:00:00.000Z"
}
```

Códigos Promocionales — Admin

Método	Ruta	Descripción
GET	/api/admin/promos	Lista todos los códigos promocionales
POST	/api/admin/promos	Crear nuevo código
PUT	/api/admin/promos/:id	Activar/desactivar código (toggle `is_active`)
DELETE	/api/admin/promos/:id	Eliminar código

Ejemplo — Crear código promocional:

```
curl -X POST http://localhost:5000/api/admin/promos \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{
    "code": "VERANO2026",
    "discount_type": "percentage",
    "discount_value": 15,
    "min_order_value": 200,
    "max_uses": 100
  }'
```

Inventario — Admin

Método	Ruta	Descripción
PUT	/api/admin/variants/:id/stock	Actualizar stock de una variante

Ejemplo:

```
curl -X PUT http://localhost:5000/api/admin/variants/5/stock \
  -H "Authorization: Bearer <TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{ "stock": 25 }'
```

Resumen de Todos los Endpoints

#	Método	Ruta	Auth	Descripción
1	GET	/	No	Mensaje de bienvenida
2	GET	/api/health	No	Estado del servidor
3	GET	/api/products	No	Listar productos (con filtros)
4	GET	/api/products/:id	No	Detalle de producto
5	GET	/api/categories	No	Listar categorías
6	POST	/api/promos/validate	No	Validar código promo
7	POST	/api/checkout/process	No	Procesar pedido
8	POST	/api/auth/login	No	Login admin
9	POST	/api/auth/verify	No	Verificar token
10	GET	/api/admin/dashboard	🔒	Dashboard métricas
11	GET	/api/admin/products	🔒	Listar productos (admin)
12	GET	/api/admin/products/:id	🔒	Detalle producto (admin)
13	POST	/api/admin/products	🔒	Crear producto
14	PUT	/api/admin/products/:id	🔒	Actualizar producto
15	DELETE	/api/admin/products/:id	🔒	Eliminar producto
16	GET	/api/admin/orders	🔒	Listar pedidos
17	GET	/api/admin/orders/:id	🔒	Detalle de pedido
18	PUT	/api/admin/orders/:id/status	🔒	Actualizar estado pedido
19	GET	/api/admin/promos	🔒	Listar promos
20	POST	/api/admin/promos	🔒	Crear promo
21	PUT	/api/admin/promos/:id	🔒	Toggle activo promo
22	DELETE	/api/admin/promos/:id	🔒	Eliminar promo
23	PUT	/api/admin/variants/:id/stock	🔒	Actualizar stock

7.3 Autenticación y Autorización

Diagrama

Detalles de la implementación:

Aspecto	Detalle
Algoritmo de hash	bcrypt con 12 rondas de salting
Token	JWT (HS256)
Expiración	24 horas
Payload JWT	`{ id, email, name }`
Almacenamiento (cliente)	`localStorage` → `zzaly_admin_token`, `zzaly_admin_user`
Verificación	Cada carga de página admin invoca `POST /api/auth/verify`
Admin por defecto	`admin@zzaly.com` / `Zzaly2026!Secure`

> [!CAUTION]
> Las credenciales de administrador por defecto deben cambiarse **inmediatamente** en producción. Utiliza el script `seed_admin.js` con las nuevas credenciales o actualízalas directamente en la base de datos.

---

8 · Base de Datos — PostgreSQL

8.1 Diagrama Entidad-Relación

Diagrama

8.2 Tipos ENUM

La base de datos define los siguientes tipos enumerados personalizados:

```
-- Estado de los pedidos
CREATE TYPE order_status AS ENUM (
  'pending',      -- ⏳ Pendiente (recién creado)
  'processing',   -- 🔄 En proceso (confirmado por admin)
  'shipped',      -- 🚚 Enviado
  'delivered',    -- ✅ Entregado
  'cancelled'     -- ❌ Cancelado
);

-- Estado de los pagos
CREATE TYPE payment_status AS ENUM (
  'pending',      -- ⏳ Pendiente
  'success',      -- ✅ Exitoso
  'failed',       -- ❌ Fallido
  'refunded'      -- 🔄 Reembolsado
);

-- Tipo de descuento
CREATE TYPE discount_type AS ENUM (
  'percentage',   -- Porcentaje (ej: 10%)
  'fixed'         -- Monto fijo (ej: Q50)
);
```

8.3 Esquema de Tablas

`zz_categories`

Almacena las categorías de productos.

Columna	Tipo	Restricciones	Descripción
`id`	`SERIAL`	`PRIMARY KEY`	Identificador auto-incremental
`name`	`VARCHAR(100)`	`NOT NULL, UNIQUE`	Nombre de la categoría
`slug`	`VARCHAR(100)`	`NOT NULL, UNIQUE`	Slug URL-friendly
`created_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Fecha de creación
`updated_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`, auto-trigger	Última actualización

`zz_products`

Catálogo principal de productos.

Columna	Tipo	Restricciones	Descripción
`id`	`SERIAL`	`PRIMARY KEY`	Identificador auto-incremental
`name`	`VARCHAR(255)`	`NOT NULL`	Nombre del producto
`slug`	`VARCHAR(255)`	`NOT NULL, UNIQUE`	Slug URL-friendly
`description`	`TEXT`	Nullable	Descripción detallada
`price`	`DECIMAL(10,2)`	`NOT NULL, CHECK >= 0`	Precio en GTQ
`category_id`	`INTEGER`	`FK → zz_categories(id), ON DELETE SET NULL`	Categoría del producto
`is_featured`	`BOOLEAN`	`DEFAULT FALSE`	Producto destacado
`is_active`	`BOOLEAN`	`DEFAULT TRUE`	Producto visible en tienda
`created_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Fecha de creación
`updated_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`, auto-trigger	Última actualización

Índices:

- `idx_products_category` → `category_id`
- `idx_products_active` → `is_active`

`zz_product_variants`

Variantes de producto (combinaciones de talla y color).

Columna	Tipo	Restricciones	Descripción
`id`	`SERIAL`	`PRIMARY KEY`	Identificador auto-incremental
`product_id`	`INTEGER`	`NOT NULL`, `FK → zz_products(id)`, `ON DELETE CASCADE`	Producto padre
`size`	`VARCHAR(10)`	`NOT NULL`	Talla (XS, S, M, L, XL)
`color`	`VARCHAR(50)`	`NOT NULL`	Nombre del color
`color_hex`	`VARCHAR(7)`	Nullable	Código hexadecimal del color
`stock`	`INTEGER`	`DEFAULT 0`, `CHECK >= 0`	Unidades disponibles
`sku`	`VARCHAR(100)`	`UNIQUE`, Nullable	Código SKU único

Restricción única compuesta: `(product_id, size, color)` — No puede existir la misma combinación talla-color para un producto.

`zz_product_images`

Galería de imágenes de cada producto.

Columna	Tipo	Restricciones	Descripción
`id`	`SERIAL`	`PRIMARY KEY`	Identificador auto-incremental
`product_id`	`INTEGER`	`NOT NULL`, `FK → zz_products(id)`, `ON DELETE CASCADE`	Producto padre
`url`	`TEXT`	`NOT NULL`	URL de la imagen
`alt_text`	`VARCHAR(255)`	Nullable	Texto alternativo (accesibilidad)
`sort_order`	`INTEGER`	`DEFAULT 0`	Orden de visualización

`zz_promo_codes`

Códigos de descuento promocionales.

Columna	Tipo	Restricciones	Descripción
`id`	`SERIAL`	`PRIMARY KEY`	Identificador auto-incremental
`code`	`VARCHAR(50)`	`NOT NULL`, `UNIQUE`	Código promocional (ej: WELCOME10)
`discount_type`	`ENUM discount_type`	`NOT NULL`	Tipo: `percentage` o `fixed`
`discount_value`	`DECIMAL(10,2)`	`NOT NULL`, `CHECK > 0`	Valor del descuento
`min_order_value`	`DECIMAL(10,2)`	`DEFAULT 0`	Pedido mínimo requerido (GTQ)
`max_uses`	`INTEGER`	Nullable (`null` = ilimitado)	Máximo de usos permitidos
`used_count`	`INTEGER`	`DEFAULT 0`	Veces que se ha utilizado
`valid_from`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Inicio de vigencia
`valid_until`	`TIMESTAMPTZ`	Nullable (`null` = sin expiración)	Fin de vigencia
`is_active`	`BOOLEAN`	`DEFAULT TRUE`	Código activo/desactivado

`zz_orders`

Pedidos de los clientes.

Columna	Tipo	Restricciones	Descripción
`id`	`UUID`	`PRIMARY KEY`, `DEFAULT uuid_generate_v4()`	Identificador único universal
`user_id`	`UUID`	Nullable	ID del usuario (`null` para invitados)
`status`	`ENUM order_status`	`DEFAULT 'pending'`	Estado actual del pedido
`subtotal`	`DECIMAL(10,2)`	`NOT NULL`	Subtotal antes de descuentos
`discount_amount`	`DECIMAL(10,2)`	`DEFAULT 0`	Monto descontado
`total`	`DECIMAL(10,2)`	`NOT NULL`	Total final cobrado
`promo_code_id`	`INTEGER`	`FK → zz_promo_codes(id)`	Código promo aplicado
`shipping_address`	`JSONB`	`NOT NULL`	Dirección de envío (objeto JSON)
`contact_email`	`VARCHAR(255)`	`NOT NULL`	Email del comprador
`contact_phone`	`VARCHAR(50)`	Nullable	Teléfono de contacto
`notes`	`TEXT`	Nullable	Notas adicionales del pedido
`created_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`	Fecha de creación
`updated_at`	`TIMESTAMPTZ`	`DEFAULT NOW()`, auto-trigger	Última actualización

`zz_order_items`

Artículos individuales dentro de cada pedido. Los datos del producto son **snapshots** (copias en el momento de la compra).

Columna	Tipo	Restricciones	Descripción
`id`	`SERIAL`	`PRIMARY KEY`	Identificador auto-incremental
`order_id`	`UUID`	`NOT NULL`, `FK → zz_orders(id)`, `ON DELETE CASCADE`	Pedido padre
`variant_id`	`INTEGER`	`FK → zz_product_variants(id)`, `ON DELETE SET NULL`	Variante comprada
`product_name`	`VARCHAR(255)`	`NOT NULL`	Nombre del producto (snapshot)
`size`	`VARCHAR(10)`	Nullable	Talla comprada
`color`	`VARCHAR(50)`	Nullable	Color comprado

quantity	INTEGER	NOT NULL, CHECK > 0	Cantidad
unit_price	DECIMAL(10,2)	NOT NULL	Precio unitario al momento de compra
subtotal	DECIMAL(10,2)	NOT NULL	Subtotal de la línea (quantity × unit_price)

> [!NOTE]
> Los campos product_name, size, color y unit_price son **copias inmutables** de los datos en el momento de la compra. Esto garantiza que los cambios futuros en el catálogo no alteren los registros históricos de pedidos.

---

zz_payments

Registros de pago asociados a pedidos.

Columna	Tipo	Restricciones	Descripción
id	UUID	PRIMARY KEY	Identificador único
order_id	UUID	NOT NULL, FK → zz_orders(id), ON DELETE CASCADE	Pedido asociado
provider	VARCHAR(50)	DEFAULT 'banco_industrial'	Proveedor de pago
transaction_id	VARCHAR(255)	UNIQUE, Nullable	ID de transacción externa
status	ENUM payment_status	DEFAULT 'pending'	Estado del pago
amount	DECIMAL(10,2)	NOT NULL	Monto cobrado
currency	VARCHAR(3)	DEFAULT 'GTQ'	Moneda (Quetzales)
gateway_response	JSONB	Nullable	Respuesta raw del gateway de pago
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación
updated_at	TIMESTAMPTZ	DEFAULT NOW(), auto-trigger	Última actualización

---

zz_admins

Usuarios administradores del sistema.

Columna	Tipo	Restricciones	Descripción
id	SERIAL	PRIMARY KEY	Identificador auto-incremental
email	VARCHAR(255)	NOT NULL, UNIQUE	Email del administrador
password_hash	VARCHAR(255)	NOT NULL	Hash bcrypt de la contraseña
name	VARCHAR(100)	NOT NULL, DEFAULT 'Admin'	Nombre del administrador
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación

8.4 Procedimiento Almacenado: `process_checkout()`

Este procedimiento almacenado es el corazón del proceso de checkout. Garantiza **integridad transaccional completa** mediante una transacción ACID con bloqueo pesimista a nivel de fila.

Flujo del procedimiento:

Diagrama

Características clave:

Característica	Detalle
Aislamiento	FOR UPDATE lock — Bloquea las filas de variantes seleccionadas hasta que la transacción termine
Atomicidad	Si algún item no tiene stock suficiente, toda la transacción se revierte (ROLLBACK)
Consistencia	El stock nunca puede quedar negativo gracias al CHECK constraint + validación en el procedimiento
Durabilidad	Una vez que COMMIT se ejecuta, todos los cambios son permanentes

---

8.5 Triggers

Todas las tablas que contienen la columna updated_at tienen asociado un trigger automático que actualiza la marca de tiempo en cada operación UPDATE:

```
CREATE OR REPLACE FUNCTION update_modified_column()
RETURNS TRIGGER AS $
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$ LANGUAGE plpgsql;

-- Se aplica a: zz_categories, zz_products, zz_orders, zz_payments
CREATE TRIGGER set_updated_at
    BEFORE UPDATE ON nombre_tabla
    FOR EACH ROW
    EXECUTE FUNCTION update_modified_column();
```

Tablas con trigger `update_modified_column()`:

- zz_categories
- zz_products
- zz_orders
- zz_payments

8.6 Datos Semilla

Los scripts de datos semilla (`seed.js` y `seed_admin.js`) populan la base de datos con información de prueba:

Categorías (6)

#	Nombre	Slug
1	Vestidos	`vestidos`
2	Blusas	`blusas`
3	Faldas	`faldas`
4	Pantalones	`pantalones`
5	Accesorios	`accesorios`
6	Calzado	`calzado`

Productos (12)

Cada producto incluye **15 variantes** (5 tallas × 3 colores): XS, S, M, L, XL en tres combinaciones de colores.

Códigos Promocionales (2)

Código	Tipo	Valor	Pedido Mínimo
`WELCOME10`	`percentage`	10%	*(ninguno)*
`DESCUENTO50`	`fixed`	Q50.00	*(depende de la configuración)*

Administrador por Defecto

Campo	Valor
Email	`admin@zzaly.com`
Contraseña	`Zzaly2026!Secure`
Nombre	`Admin`

---

9 · Frontend — React SPA

9.1 Enrutamiento

La aplicación utiliza **React Router v7** para el enrutamiento del lado del cliente. Las rutas se dividen en dos grupos: públicas (tienda) y administrativas (panel).

Ruta	Componente	Tipo	Descripción
`/`	`Home`	🌐 Pública	Página de inicio con carrusel y productos destacados
`/nosotros`	`Nosotros`	🌐 Pública	Información sobre la marca
`/catalogo`	`Catalog`	🌐 Pública	Catálogo con filtros avanzados
`/carrito`	`Cart`	🌐 Pública	Carrito de compras y checkout
`/admin/login`	`AdminLogin`	🔒 Admin	Formulario de inicio de sesión
`/admin`	`AdminLayout → AdminDashboard`	🔒 Admin	Dashboard principal
`/admin/dashboard`	`AdminDashboard`	🔒 Admin	Dashboard con métricas
`/admin/productos`	`AdminProducts`	🔒 Admin	CRUD de productos
`/admin/pedidos`	`AdminOrders`	🔒 Admin	Gestión de pedidos
`/admin/promos`	`AdminPromos`	🔒 Admin	Gestión de códigos promo

Diagrama

> [!NOTE]
> Todas las rutas admin (excepto login) están envueltas por `AdminLayout`, que verifica la autenticación JWT antes de renderizar el contenido. Si el token es inválido o no existe, redirige automáticamente a `/admin/login`.

9.2 Gestión de Estado

CartContext (Carrito de Compras)

El estado global del carrito se maneja mediante la **Context API de React**, implementada en `CartContext.jsx`.

Diagrama

Funcionalidades del CartContext:

Propiedad/Método	Tipo	Descripción
`items`	`Array`	Lista de artículos en el carrito
`addItem(product, variant, qty)`	`Function`	Agrega un artículo o incrementa cantidad si ya existe
`removeItem(variantId)`	`Function`	Elimina un artículo del carrito
`updateQuantity(variantId, qty)`	`Function`	Actualiza la cantidad de un artículo
`clearCart()`	`Function`	Vacía el carrito completamente
`totalItems`	`Number`	Total de unidades en el carrito (computado)
`totalPrice`	`Number`	Precio total del carrito en GTQ (computado)
`toast`	`Object`	Sistema de notificaciones visuales

Persistencia:

- **Clave:** `zzaly_cart` en `localStorage`
- **Formato:** JSON serializado del array `items`
- **Comportamiento:** Se sincroniza automáticamente en cada cambio de estado

Autenticación Admin

Clave localStorage	Contenido	Propósito
`zzaly_admin_token`	Token JWT (string)	Autenticación en peticiones API
`zzaly_admin_user`	Objeto JSON `{ id, email, name }`	Información del admin para la UI

9.3 Sistema de Diseño

El sistema de diseño se encuentra definido en `index.css` con más de **3,000 líneas** de CSS personalizado.

Paleta de Colores

Variable CSS	Color	Hex	Uso
`--cream`	Crema	`#F6F2EF`	Fondo principal
`--sand`	Arena	`#EBDAC9`	Fondos secundarios, tarjetas
`--rose-tan`	Rosa tostado	`#BC967F`	Acentos, botones hover
`--espresso`	Espresso	`#5D4032`	Texto principal, bordes

Tipografía

Fuente	Tipo	Uso	Fuente
Cormorant Garamond	Serif (display)	Títulos, encabezados, logotipo	Google Fonts
Outfit	Sans-serif (body)	Texto de cuerpo, botones, UI general	Google Fonts

Breakpoints Responsive

Breakpoint	Ancho	Target
Desktop	> 1024px	Pantallas de escritorio
Tablet	≤ 1024px	Tablets y pantallas medianas
Mobile	≤ 768px	Teléfonos en modo landscape
Mobile Small	≤ 480px	Teléfonos en modo portrait

Efectos Visuales

Efecto	Descripción	Ubicación
Glassmorphism	Navbar con efecto de vidrio translúcido (`backdrop-filter: blur()`)	`Navbar.jsx`
Parallax	Fondos con efecto de desplazamiento diferencial	`Home.jsx`
Scroll Reveal	Animaciones de entrada al hacer scroll (hook personalizado)	`useScrollReveal.js`
Loading Skeletons	Placeholders animados durante la carga de datos	Varias páginas

---

9.4 Componentes Principales

Componente	Archivo	Descripción
Navbar	`components/Navbar.jsx`	Barra de anuncios superior + navegación con efecto glassmorphism + mega menú desplegable para categorías + menú hamburguesa en móvil + icono de carrito con contador
HeroCarousel	`components/HeroCarousel.jsx`	Carrusel principal de la página de inicio usando Swiper.js. 3 slides con efecto fade y reproducción automática
ProductCard	`components/ProductCard.jsx`	Tarjeta de producto con carrusel de imágenes, badge de "Destacado", botón de vista rápida y puntos de color disponibles
ProductDetail	`components/ProductDetail.jsx`	Modal de detalle con galería de imágenes, selectores de talla/color, indicador de stock y botón de agregar al carrito
CatalogFilters	`components/CatalogFilters.jsx`	Panel de filtros del catálogo: categoría, talla, color y rango de precio. Comportamiento de toggle (seleccionar/deseleccionar)
Footer	`components/Footer.jsx`	Pie de página con layout de 4 columnas: navegación, categorías, contacto y redes sociales

---

10 · Seguridad

La aplicación implementa múltiples capas de seguridad para proteger tanto la API como los datos de los usuarios:

Diagrama

Medidas de Seguridad Implementadas

#	Medida	Implementación	Protege contra
1	**Helmet**	Cabeceras HTTP de seguridad con `crossOriginResourcePolicy: 'cross-origin'`	XSS, clickjacking, MIME sniffing
2	**CORS**	Orígenes configurables (whitelist dinámica)	Solicitudes cross-origin no autorizadas
3	**Rate Limiting General**	200 peticiones / 15 minutos para `/api/*`	DDoS, abuso de API
4	**Rate Limiting Login**	10 peticiones / 15 minutos para `/api/auth/login`	Fuerza bruta en contraseñas
5	**JWT**	Tokens firmados con HS256, expiración 24h	Acceso no autorizado a rutas admin
6	**bcrypt**	Hash con 12 rondas de salting	Robo de contraseñas en texto plano
7	**Queries parametrizadas**	Todas las consultas SQL usan placeholders `$1, $2, ...`	Inyección SQL
8	**Row-level locking**	`SELECT ... FOR UPDATE` en checkout	Condiciones de carrera en el inventario
9	**Body size limit**	`express.json({ limit: '10mb' })`	Payloads excesivamente grandes
10	**CHECK constraints**	`price >= 0, stock >= 0, quantity > 0`	Datos inválidos en la BD

> [!WARNING]
> **Recomendaciones adicionales para producción:**
>
> - Cambiar el `JWT_SECRET` por defecto por una clave de al menos 64 caracteres aleatorios
> - Cambiar las credenciales del administrador por defecto
> - Habilitar HTTPS en el backend (gestionado automáticamente por Render.com)
> - Configurar `FRONTEND_URL` para restringir CORS a un origen específico
> - Considerar implementar CSRF tokens para formularios críticos
> - Añadir logs de auditoría para acciones administrativas

---

11 · Despliegue a Producción

Arquitectura de Despliegue

Diagrama

Backend — Render.com

El backend se despliega en Render.com usando el archivo `render.yaml` de la raíz del proyecto:

```yaml
# render.yaml
services:
  - type: web
    name: ecommerce-backend
    env: node
    plan: free
    buildCommand: cd server && npm install
    startCommand: cd server && node index.js
    envVars:
      - key: DATABASE_URL
        value: <tu-connection-string-de-supabase>
      - key: JWT_SECRET
        value: <tu-clave-secreta-segura>
      - key: FRONTEND_URL
        value: <url-de-tu-frontend>
```

Características del despliegue en Render:

Aspecto	Detalle
Plan	Free tier
Auto-deploy	Sí (conectado a repositorio Git)
HTTPS	Automático (Let's Encrypt)
Región	Configurable
Cold starts	El free tier puede tener latencia inicial (~30s)

> [!TIP]
> En el free tier de Render, el servidor se duerme después de 15 minutos de inactividad. La primera petición después de la inactividad puede tardar 30-50 segundos. Para producción real, considerar el plan pagado.

Base de Datos — Supabase

Aspecto	Detalle
Servicio	Supabase (PostgreSQL gestionado)
Conexión	SSL habilitado
Formato	`postgresql://user:pass@host:port/db`
Pooling	Conexión a través de Supabase Pooler (puerto 6543)

Frontend — Hosting Estático

El frontend puede desplegarse en cualquier servicio de hosting estático:

```bash
# Generar build de producción
cd client
npm run build
# Los archivos estáticos se generan en client/dist/
```

Servicio	Comando de Deploy	Notas
Netlify	Arrastrar carpeta `dist/` o CLI	Soportado en CORS por defecto
Vercel	`vercel --prod`	Framework preset: Vite
GitHub Pages	Configurar en repositorio	Requiere base path

> [!IMPORTANT]
> Asegúrate de configurar `VITE_API_URL` con la URL del backend en Render **antes** de ejecutar `npm run build`. Esta variable se incorpora al build en tiempo de compilación y no puede cambiarse después.

---

12 · Guía de Contribución

Convenciones de Código

Aspecto	Convención
Idioma del código	Inglés para nombres de variables y funciones
Idioma de la UI	Español para todo texto visible al usuario
Formato CSS	Variables CSS globales en `:root`, clases BEM-inspired
Componentes React	Functional components con hooks
Nombrado de archivos	PascalCase para componentes, camelCase para hooks/utils
Rutas API	kebab-case, plurales para recursos
Tablas BD	Prefijo `zz_`, snake_case

Flujo de Trabajo

```
# 1. Crear rama de feature
git checkout -b feature/nueva-funcionalidad

# 2. Desarrollar y probar localmente
npm start          # Backend
npm run dev        # Frontend

# 3. Commit con mensaje descriptivo
git add .
git commit -m "feat: agregar filtro de precio al catálogo"

# 4. Push y crear Pull Request
git push origin feature/nueva-funcionalidad
```

Scripts Disponibles

Servidor

```
cd server
npm start          # Iniciar servidor (node index.js)
node db/seed.js    # Ejecutar datos semilla de productos
node db/seed_admin.js  # Crear admin por defecto
```

Cliente

```
cd client
npm run dev        # Servidor de desarrollo (Vite)
npm run build      # Build de producción
npm run preview    # Preview del build local
```

---

13 · Solución de Problemas

Errores Comunes

Problema	Causa Probable	Solución
`ECONNREFUSED` al conectar a la BD	PostgreSQL no está corriendo o credenciales incorrectas	Verificar que PostgreSQL esté activo y revisar variables `DB_*` en `.env`
CORS error en el navegador	`FRONTEND_URL` no configurado	Agregar la URL del frontend a `FRONTEND_URL` en `server/.env`
401 en rutas admin	Token expirado o inválido	Cerrar sesión y volver a iniciar sesión
Productos no cargan	`VITE_API_URL` incorrecto	Verificar que apunte al backend correcto en `client/.env`
`rate limit exceeded`	Demasiadas peticiones	Esperar 15 minutos o ajustar el límite en desarrollo
Checkout falla con 409	Stock insuficiente	Verificar stock de las variantes desde el panel admin
`MODULE_NOT_FOUND`	Dependencias no instaladas	Ejecutar `npm install` en `/server` y `/client`
Build falla en Render	Variables de entorno no configuradas	Verificar todas las env vars en el dashboard de Render

Verificación del Sistema

```
# 1. Verificar que el backend responde
curl http://localhost:5000/api/health

# 2. Verificar conexión a la base de datos
curl http://localhost:5000/api/categories

# 3. Verificar autenticación
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@zzaly.com","password":"Zzaly2026!Secure"}'

# 4. Verificar que el frontend se conecta al backend
# Abrir http://localhost:5173 y verificar que los productos cargan
```

Zzaly E-commerce · Documentación Técnica v1.0.0

Desarrollado con ❤ para el mercado guatemalteco

© 2026 Zzaly — Todos los derechos reservados

Guía de Usuario — Zzaly E-Commerce

Plataforma de Moda Femenina · Guatemala GT

Versión 1.0 · Mayo 2026

Tabla de Contenidos

Parte 1 — Guía para Clientes

1. Introducción
2. Navegación del Sitio
3. Página de Inicio
4. Catálogo de Productos
5. Carrito de Compras
6. Nosotros

Parte 2 — Guía de Administrador

7. Acceso al Panel de Administración
8. Dashboard
9. Gestión de Productos
10. Gestión de Pedidos
11. Gestión de Promociones
12. Navegación del Panel Admin

Parte 3 — Preguntas Frecuentes

13. Preguntas Frecuentes (FAQ)

PARTE 1 — GUÍA PARA CLIENTES

---

1. Introducción

¡Bienvenida a **Zzaly**! ✨

Zzaly es tu tienda en línea de **moda femenina** diseñada especialmente para el mercado guatemalteco. Aquí encontrarás prendas y accesorios cuidadosamente seleccionados que combinan elegancia atemporal con las últimas tendencias de la moda.

Dato	Detalle
Tipo de tienda	Moda femenina contemporánea
Mercado	Guatemala
Moneda	Quetzales guatemaltecos (Q)
Sitio web	Frontend desplegado en la web
Contacto	hola@zzaly.com

> [!TIP]
> No necesitas crear una cuenta para comprar. Zzaly funciona con **checkout como invitado**, lo que significa que puedes comprar de forma rápida y sencilla sin necesidad de registrarte.

¿Qué encontrarás en Zzaly?

- **Vestidos** — Para cada ocasión especial
- **Blusas** — Esenciales del guardarropa diario
- **Pantalones** — Comodidad y estilo
- **Abrigos** — Para la temporada perfecta
- **Accesorios** — El toque final de tu outfit

---

2. Navegación del Sitio

La navegación de Zzaly está diseñada para ser intuitiva y elegante. A continuación, te explicamos cada elemento de la interfaz.

2.1 Barra de Anuncios (Announcement Bar)

En la parte **superior** del sitio encontrarás una barra con mensajes promocionales que se desplazan automáticamente de forma continua. Los mensajes incluyen:

Emoji	Mensaje
✨	**ENVÍO GRATIS** en compras mayores a Q500
🎁	**NUEVA COLECCIÓN DISPONIBLE**
💳	**PAGOS SEGUROS**

> [!NOTE]
> La barra de anuncios se desplaza automáticamente con un efecto *marquee* horizontal para que puedas ver todas las promociones activas sin necesidad de interactuar.

2.2 Barra de Navegación Principal

Debajo de la barra de anuncios encontrarás la **barra de navegación** que contiene:

Diagrama

Enlaces de navegación:

Enlace	Ruta	Descripción
Inicio	`/`	Página principal con colecciones destacadas
Nosotros	`/nosotros`	Historia y misión de Zzaly
Catálogo	`/catalogo`	Todos los productos con filtros
Carrito	`/carrito`	Tu carrito de compras

Mega Menú del Catálogo

Al pasar el cursor sobre **"Catálogo"**, se despliega un **mega menú** elegante que muestra:

- **Lado izquierdo**: Lista de todas las categorías disponibles (Vestidos, Blusas, Pantalones, Abrigos, Accesorios) + enlace "Ver Todo →"
- **Lado derecho**: Imagen promocional de la nueva temporada

> [!TIP]
> Haz clic en cualquier categoría del mega menú para ir directamente al catálogo ya filtrado por esa categoría. ¡Es la forma más rápida de encontrar lo que buscas!

Badge del Carrito

El ícono del carrito (🛒) muestra un **badge numérico** cuando tienes productos agregados. Este número indica la cantidad total de artículos en tu carrito.

2.3 Menú Móvil

En **dispositivos móviles** (pantallas menores a 768px), la navegación se adapta de la siguiente manera:

1. Los enlaces de navegación se ocultan
2. Aparece un **botón hamburguesa** (☰) en la esquina derecha
3. Al presionarlo, se abre un **panel deslizable** desde el lado derecho de la pantalla

Contenido del menú móvil:

- Enlace a Inicio
- Enlace a Nosotros
- Sección expandida de **Catálogo** con todas las categorías
- Enlace al **Carrito** (con cantidad entre paréntesis si hay productos)

> [!NOTE]
> El menú móvil se cierra automáticamente al navegar a cualquier página. También puedes cerrarlo presionando el botón hamburguesa nuevamente.

2.4 Comportamiento al Desplazarse (Scroll)

La barra de navegación tiene un comportamiento inteligente:

Acción	Comportamiento
Scroll hacia abajo (>300px)	La barra se **oculta** suavemente
Scroll hacia arriba	La barra **reaparece** con estilo sólido
Parte superior de la página	La barra es **transparente** (solo en la página de Inicio)
Scroll > 60px	La barra cambia a fondo **sólido**

2.5 Pie de Página (Footer)

El pie de página contiene información organizada en columnas:

Columna	Contenido
Marca (zzaly)	Descripción de la marca: "Moda femenina contemporánea con un toque de elegancia atemporal"
Tienda	Novedades, Vestidos, Blusas, Pantalones, Accesorios
Información	Sobre Nosotras, Envíos, Devoluciones, Guía de Tallas
Contacto	hola@zzaly.com, Instagram, Pinterest, WhatsApp

Al final se muestra: *© 2026 zzaly. Todos los derechos reservados.*

🏠 3. Página de Inicio

La página de inicio es la vitrina principal de Zzaly. Está compuesta por las siguientes secciones:

3.1 Carrusel Hero

Lo primero que verás al entrar es un impresionante **carrusel a pantalla completa** con tres diapositivas:

#	Título	Subtítulo	Botón
1	**Nueva Colección Primavera**	Descubre las últimas tendencias en moda femenina. Piezas únicas diseñadas para ti.	Explorar Colección
2	**Elegancia sin Esfuerzo**	Nuestra selección de prendas premium combina confort y estilo para tu día a día.	Ver Novedades
3	**Viste tu Esencia**	Encuentra tu estilo con nuestra colección cuidadosamente curada para la mujer moderna.	Comprar Ahora

Características del carrusel:

- 🔄 **Autoplay**: Cambia automáticamente cada 5 segundos
- 🎭 **Efecto fade**: Transición suave entre diapositivas
- ⏱️ **Velocidad**: 1 segundo por transición

- ○ **Indicadores**: Puntos clickeables en la parte inferior
- ∞ **Loop infinito**: Se repite continuamente

> [!TIP]
> Cada diapositiva tiene un botón que te lleva directamente al catálogo. ¡Haz clic para comenzar a explorar!

3.2 Ticker de Promociones

Debajo del carrusel hero encontrarás una **banda animada** con mensajes que se desplazan continuamente:

> *NUEVA COLECCIÓN 2026 ★ ENVÍO GRATIS +Q500 ★ PRENDAS EXCLUSIVAS ★ MODA FEMENINA ★ ELEGANCIA ATEMPORAL*

3.3 Sección de Categorías

Cuatro tarjetas visualmente atractivas que te permiten explorar cada categoría directamente:

Categoría	Etiqueta	Imagen
👗 **Vestidos**	Colección 2026	Imagen elegante con overlay
👚 **Blusas**	Esenciales	Imagen con estilo minimalista
🧥 **Abrigos**	Temporada	Imagen de temporada
💍 **Accesorios**	Complementos	Imagen de accesorios

Cada tarjeta tiene:

- Una **imagen de fondo** a pantalla completa
- Un **overlay** con el nombre de la categoría
- Una etiqueta descriptiva
- Un enlace **"Explorar →"** al pasar el cursor
- **Animación de aparición** al hacer scroll (reveal on scroll)

3.4 Banner Parallax

Un banner a ancho completo con **efecto parallax** que muestra:

- Etiqueta: *NUEVA TEMPORADA*
- Título: **"Elegancia que Define tu Estilo"**
- Descripción: *"Cada pieza cuenta una historia. Descubre prendas diseñadas para la mujer que marca tendencia."*
- Botón: **Descubrir Colección**

3.5 Productos Destacados

Sección titulada **"Lo Más Deseado — Productos Destacados"** que muestra automáticamente los productos marcados como destacados en la tienda.

- Se cargan dinámicamente desde el servidor
- Mientras cargan, se muestran **skeletons** (placeholders animados)
- Cada producto se muestra como tarjeta interactiva (ver sección de Catálogo)
- Al final: botón **"Ver Todo el Catálogo"**

3.6 Banner Lookbook

Un banner dividido en dos mitades:

- **Izquierda**: Imagen inspiracional
- **Derecha**: Texto "Inspírate con Nuestros Looks" + descripción + botón **"Ver Lookbook"**

3.7 Segundo Ticker

Otra banda animada oscura con mensajes:

> *CALIDAD PREMIUM ✦ DISEÑO EXCLUSIVO ✦ MODA SOSTENIBLE ✦ ENVÍO RÁPIDO ✦ ATENCIÓN PERSONALIZADA*

3.8 Newsletter

Sección para suscribirse al boletín de Zzaly:

- Título: **"Únete a la Familia zzaly"**
- Descripción: *"Suscríbete y recibe un 15% de descuento en tu primera compra, además de acceso exclusivo a nuevas colecciones."*
- Campo de email (placeholder: `tu@email.com`)
- Botón **"Suscribirme →"**

> [!TIP]
> Al suscribirte al newsletter, recibirás un código de **15% de descuento** para tu primera compra. ¡No te lo pierdas!

🛍 4. Catálogo de Productos

El catálogo es donde encontrarás todos los productos disponibles en Zzaly. Puedes acceder de varias formas:

- Desde el enlace **"Catálogo"** en la barra de navegación
- Desde el **mega menú** (seleccionando una categoría específica)
- Desde las **tarjetas de categorías** en la página de inicio
- Desde cualquier **botón de acción** (CTA) en los banners

4.1 Estructura de la Página

Diagrama

La página se divide en:

- **Sidebar izquierdo**: Panel de filtros (se oculta en móvil)
- **Área principal**: Encabezado con título + conteo de productos, grilla de tarjetas
- **Botón de filtros móvil**: Aparece solo en pantallas pequeñas para mostrar/ocultar filtros

4.2 Filtros Disponibles

Filtro	Opciones	Tipo de Control
Categoría	Vestidos, Blusas, Pantalones, Faldas, Abrigos, Accesorios	Radio buttons
Talla	XS, S, M, L, XL	Radio buttons
Color	Beige, Marrón, Rosa, Negro, Blanco	Círculos de color (swatches)
Precio	Mínimo (Q) — Máximo (Q)	Campos numéricos

Cómo usar los filtros:

1. **Seleccionar**: Haz clic en cualquier opción para activar el filtro
2. **Deseleccionar**: Haz clic nuevamente en la misma opción para quitar el filtro
3. **Limpiar todos**: Presiona el botón **"✕ Limpiar filtros"** que aparece cuando hay filtros activos
4. **Combinación**: Puedes combinar múltiples filtros simultáneamente

> [!NOTE]
> Los filtros se aplican **automáticamente** al seleccionarlos. No necesitas presionar ningún botón de "Buscar" — los resultados se actualizan en tiempo real.

> [!TIP]
> Si llegas al catálogo desde una categoría específica (por ejemplo, desde el mega menú seleccionando "Vestidos"), el filtro de categoría se seleccionará automáticamente. Puedes cambiarlo o limpiar los filtros en cualquier momento.

4.3 Tarjetas de Producto

Cada producto se muestra como una **tarjeta interactiva** con los siguientes elementos:

Elemento	Descripción
Imagen	Imagen principal del producto. Si hay múltiples imágenes, se muestra un mini carrusel con indicadores
Badge "Destacado"	Aparece en la esquina si el producto está marcado como destacado (★)
Vista Rápida	Botón que aparece al pasar el cursor sobre la imagen
Categoría	Nombre de la categoría en texto pequeño
Nombre	Nombre del producto
Precio	Precio en Quetzales (Q)
Puntos de Color	Círculos pequeños que indican los colores disponibles

4.4 Vista Detallada del Producto (Modal)

Al hacer clic en una tarjeta de producto o en el botón **"Vista Rápida"**, se abre un **modal** elegante con toda la información del producto:

Lado izquierdo — Galería de Imágenes:

- Carrusel con todas las imágenes del producto
- Navegación con flechas (anterior/siguiente)
- Indicadores de puntos clickeables
- Loop infinito

Lado derecho — Información del Producto:

1. **Categoría** del producto
2. **Nombre** del producto
3. **Precio** en Quetzales
4. **Descripción** detallada
5. **Selector de Talla**: Botones con las tallas disponibles (XS, S, M, L, XL, etc.)
6. **Selector de Color**: Círculos de color con los colores disponibles
7. **Botón "Agregar al Carrito"**

Pasos para agregar un producto:

1. Haz clic en la tarjeta del producto o en **"Vista Rápida"**
2. En el modal, selecciona una **talla** haciendo clic en el botón correspondiente
3. Selecciona un **color** haciendo clic en el círculo del color deseado
4. Presiona el botón **"Agregar al Carrito"**
5. El botón cambiará a **"✓ Añadido al Carrito"** durante 2 segundos como confirmación
6. También verás un **toast** (notificación flotante) confirmando la acción

> [!IMPORTANT]
> Debes seleccionar **tanto la talla como el color** antes de poder agregar el producto al carrito. Si no los seleccionas, el botón estará deshabilitado y verás el mensaje: *"Selecciona talla y color para continuar"*.

Cerrar el modal:

- Haz clic en el botón ✕ en la esquina superior derecha

- Haz clic fuera del modal (en el overlay oscuro)

🛒 5. Carrito de Compras

El carrito te permite revisar, modificar y procesar tu pedido.

5.1 Acceder al Carrito

Puedes acceder al carrito de las siguientes formas:

- Haciendo clic en el **ícono del carrito** (🛒) en la barra de navegación
- A través del enlace **"Carrito"** en el menú móvil

5.2 Carrito Vacío

Si tu carrito está vacío, verás:

- Un ícono de bolsa de compras
- El mensaje **"Tu carrito está vacío"**
- La descripción: *"Explora nuestra colección y encuentra prendas increíbles"*
- Un botón **"Ir al Catálogo"** para comenzar a comprar

5.3 Productos en el Carrito

Cada producto en el carrito muestra:

Elemento	Detalle
Imagen	Miniatura del producto
Nombre	Nombre completo del producto
Variante	Talla y Color seleccionados (ej. "Talla: M · Color: Negro")
Control de cantidad	Botones **−** y **+** para ajustar cantidad
Precio	Precio total por esa línea (precio unitario × cantidad)
Eliminar	Botón "Eliminar" para quitar el producto

Controles de cantidad:

- Presiona **−** para reducir la cantidad
- Presiona **+** para aumentar la cantidad
- Si la cantidad llega a 0, el producto se elimina automáticamente

5.4 Códigos Promocionales

Zzaly acepta **códigos de descuento** que puedes aplicar a tu pedido.

Cómo aplicar un código:

1. Ubica la sección **"¿Tienes un código de descuento?"** en el resumen del carrito
2. Escribe tu código en el campo de texto (ej. `WELCOME10`)
3. Presiona el botón **"Aplicar"**
4. Si el código es válido ☑: Verás el mensaje *"Código aplicado: [CÓDIGO]"* en color verde
5. Si el código es inválido ❌: Verás un mensaje de error en color rojo

Códigos de ejemplo:

Código	Tipo	Descuento
`WELCOME10`	Porcentaje	10% de descuento sobre el subtotal
`DESCUENTO50`	Monto fijo	Q50.00 de descuento directo

> [!NOTE]
> Los códigos promocionales pueden tener condiciones como un **pedido mínimo** o un **límite de usos**. Si tu pedido no cumple los requisitos, se mostrará un error indicando la razón.

5.5 Resumen del Pedido

Línea	Detalle
Subtotal	Suma de todos los productos (muestra cantidad de artículos)
Descuento	Solo aparece si hay un código promo aplicado (en color rojo)
Envío	Siempre muestra "Gratis"
Total	Monto final a pagar

5.6 Procesar el Pedido (Checkout)

Al presionar el botón **"Proceder al Pago"**:

1. El botón cambia a **"Procesando..."** y se deshabilita temporalmente
2. Se envía la orden al servidor con todos los datos del carrito
3. El sistema verifica el stock de cada producto
4. Se crea la orden y se reserva un ID de pago
5. Se muestra una confirmación con el **ID de la orden** y el **ID de la transacción de pago**

> [!IMPORTANT]
> El sistema de pagos está preparado para integración con **Banco Industrial**. Actualmente, el checkout procesa la orden y genera los identificadores necesarios para cuando la pasarela de pago esté activa.

> [!WARNING]
> Si algún producto no tiene suficiente stock para la cantidad solicitada, el checkout **fallará** y verás un mensaje de error. En ese caso, ajusta las cantidades y vuelve a intentar.

---

6. Nosotros

La página **"Nosotros"** (`/nosotros`) presenta la historia y valores de Zzaly.

Contenido:

Hero Banner

- Imagen hero a 60% de la altura de la pantalla
- Título: **"Nuestra Historia"**
- Subtítulo: *"Elevando la moda femenina desde 2026"*

Sección "Sobre Zzaly"

Nuestra Misión:

> *En Zzaly, creemos que la moda es más que ropa; es una forma de expresión personal, empoderamiento y confianza. Nacimos de la idea de proporcionar prendas exclusivas que combinan a la perfección elegancia atemporal con tendencias modernas.*
>
> *Cada diseño en nuestra tienda es cuidadosamente seleccionado y curado, pensado para mujeres modernas que no quieren sacrificar la comodidad por el estilo.*

Compromiso con la Calidad:

> *Prestamos atención a los detalles infinitos. Desde la caída de una blusa de seda natural hasta la estructura de un blazer entallado, nuestra calidad reside en la selección de materiales premium y acabados impecables.*
>
> *Nuestro equipo está dedicado a ofrecer una experiencia premium no solo en nuestros productos, sino también en el servicio que te brindamos a través de esta plataforma interactiva y segura.*

PARTE 2 — GUÍA DE ADMINISTRADOR

---

7. Acceso al Panel de Administración

El panel de administración de Zzaly permite gestionar todos los aspectos de la tienda: productos, pedidos y promociones.

7.1 Datos de Acceso

Dato	Valor
URL de login	`/admin/login`
Email por defecto	`admin@zzaly.com`
Contraseña por defecto	`Zzaly2026!Secure`
Duración de sesión	24 horas (token JWT)

> [!CAUTION]
> Las credenciales anteriores son las **credenciales por defecto**. Se recomienda encarecidamente cambiar la contraseña del administrador después del primer inicio de sesión para mayor seguridad.

7.2 Proceso de Login

Diagrama

Paso a paso:

1. Navega a la URL `/admin/login` en tu navegador
2. Verás el formulario de inicio de sesión con el logo de Zzaly y el texto "Panel de Administración"
3. Ingresa tu **correo electrónico** en el campo correspondiente
4. Ingresa tu **contraseña**
5. Haz clic en el botón **"Iniciar Sesión"**
6. Si las credenciales son correctas, serás redirigido al **Dashboard**
7. Si son incorrectas, verás un mensaje de error

> [!NOTE]
> En la parte inferior del formulario verás un indicador de **"Conexión segura"** con un ícono de candado, confirmando que la comunicación es encriptada.

7.3 Sesión y Autenticación

- La sesión utiliza **tokens JWT** (JSON Web Tokens) con expiración de **24 horas**
- El token se almacena en `localStorage` del navegador
- Cada vez que accedes al panel, se **verifica** la validez del token
- Si el token expira o es inválido, serás redirigido automáticamente al login

7.4 Cerrar Sesión

Para cerrar sesión:

1. Ubica el botón **"Cerrar Sesión"** en la parte inferior del sidebar
2. Haz clic en él
3. Se eliminarán tus datos de sesión y serás redirigido al login

---

8. Dashboard

El Dashboard es la página principal del panel de administración. Ofrece una **vista general rápida** del estado de tu tienda.

8.1 Tarjetas de Estadísticas

En la parte superior encontrarás **4 tarjetas** con las métricas más importantes:

Tarjeta	Ícono	Dato que muestra
Productos		Número total de productos en inventario
Pedidos		Número total de pedidos recibidos
Ingresos Totales		Suma total de ingresos en Quetzales (Q)
Alertas de Stock Bajo	⚠ / ☑	Cantidad de variantes con stock ≤ 5 unidades

> [!NOTE]
> La tarjeta de **"Alertas de Stock Bajo"** cambia de color según el estado:
>
> - **Verde** con ☑ si no hay alertas (stock saludable)
> - **Rosa** con ⚠ si hay variantes con stock bajo

8.2 Pedidos Recientes

Debajo de las estadísticas, una **tabla** muestra los pedidos más recientes:

Columna	Descripción
Email	Correo electrónico del cliente
Total	Monto total del pedido (en Q)
Estado	Badge colorizado del estado actual
Fecha	Fecha de creación del pedido

8.3 Acciones Rápidas

Un panel con botones de acceso rápido a las secciones principales:

Botón	Destino
Gestionar Productos	`/admin/productos`
Ver Pedidos	`/admin/pedidos`
Códigos Promo	`/admin/promos`

---

9. Gestión de Productos

La sección de productos te permite ver, crear, editar y eliminar los productos de tu tienda.

Ruta: `/admin/productos`

9.1 Lista de Productos

Al entrar verás:

- **Encabezado**: Título "Productos" + contador (ej. "12 productos en inventario")
- **Botón "✚ Nuevo Producto"**: Para crear un nuevo producto
- **Grilla de tarjetas**: Cada producto como una tarjeta visual

Cada tarjeta muestra:

Elemento	Detalle
Imagen	Primera imagen del producto
Badge ★ Destacado	Si el producto está marcado como destacado
Nombre	Nombre del producto
Categoría	Nombre de la categoría
Precio	Precio en Quetzales (Q)
Stock total	Suma del stock de todas las variantes
Botón ✏ Editar	Abre el modal de edición
Botón 🗑 Eliminar	Elimina el producto (con confirmación)

9.2 Crear un Nuevo Producto

Haz clic en **"✚ Nuevo Producto"** para abrir el modal de creación.

Campos del formulario:

Campo	Tipo	Obligatorio	Descripción
Nombre	Texto	☑	Nombre del producto. Al escribirlo, se genera automáticamente el **slug**
Slug (URL)	Texto	☑	Identificador URL-friendly. Se auto-genera pero es editable
Precio (Q)	Número	☑	Precio en Quetzales con hasta 2 decimales
Categoría	Desplegable	☑	Selecciona de las categorías existentes
Descripción	Texto largo	☑	Descripción completa del producto
Producto Destacado	Checkbox	—	Marca el producto para que aparezca en la sección destacados
Activo	Checkbox	—	Determina si el producto es visible en la tienda

> [!TIP]
> El **slug** se genera automáticamente al escribir el nombre. Por ejemplo, si escribes *"Vestido Floral Primavera"*, el slug será `vestido-floral-primavera`. Puedes editarlo manualmente si lo necesitas.

Variantes

Las variantes representan las combinaciones de talla, color y stock disponibles para un producto.

Cada variante tiene:

Campo	Tipo	Descripción
Talla	Texto	Ej. XS, S, M, L, XL
Color	Texto	Nombre del color (ej. "Negro", "Rosa")
Hex del color	Selector de color	Código hexadecimal para la muestra visual
Stock	Número	Cantidad disponible de esta combinación

Acciones de variantes:

- ✚ **Variante**: Agrega una nueva fila de variante (por defecto: L, Blanco, #ffffff, 5)
- ✕: Elimina una variante específica (disponible cuando hay más de una)

> [!IMPORTANT]
> Cada producto debe tener **al menos una variante**. Las variantes determinan qué combinaciones de talla y color están disponibles para los clientes.

Imágenes

Las imágenes se agregan mediante **URLs externas**.

- **Campo URL**: Ingresa la URL completa de la imagen
- ✚ **Imagen**: Agrega un campo adicional para otra imagen
- ✕: Elimina una URL de imagen (disponible cuando hay más de una)

> [!TIP]
> La primera imagen de la lista será la imagen principal del producto que se muestra en las tarjetas del catálogo. ¡Asegúrate de que sea la mejor!

Guardar

Presiona **"Crear Producto"** para guardar. Si hay un error, se mostrará un mensaje con los detalles.

9.3 Editar un Producto

1. Ubica el producto en la grilla
2. Haz clic en **"✏ Editar"**
3. Se abre el mismo modal de creación, pero con todos los campos **pre-llenados** con los datos actuales
4. Realiza los cambios necesarios
5. Presiona **"Guardar Cambios"**

9.4 Eliminar un Producto

1. Ubica el producto en la grilla
2. Haz clic en **"🗑 Eliminar"**
3. Aparecerá un **diálogo de confirmación**: *"¿Estás seguro de eliminar este producto?"*
4. Confirma para eliminar o cancela para mantener el producto

> [!CAUTION]
> La eliminación de un producto es **permanente** y también eliminará todas sus variantes e imágenes asociadas. Esta acción no se puede deshacer.

🛒 10. Gestión de Pedidos

La sección de pedidos te permite ver todos los pedidos recibidos, cambiar su estado y ver los detalles de cada uno.

Ruta: `/admin/pedidos`

10.1 Tabla de Pedidos

La vista principal muestra una tabla con todos los pedidos:

Columna	Descripción

ID	UUID del pedido truncado a 8 caracteres (formato monoespaciado)
Cliente	Email de contacto del cliente
Total	Monto total en Quetzales (Q) con formato de 2 decimales
Estado	Badge colorizado con el estado actual
Fecha	Fecha de creación en formato local guatemalteco
Acciones	Selector desplegable para cambiar el estado

10.2 Estados de Pedido

Los pedidos pueden tener los siguientes estados:

Estado	Nombre en español	Descripción
`pending`	**Pendiente**	Pedido recibido, sin procesar
`processing`	**Procesando**	Pedido en preparación
`shipped`	**Enviado**	Pedido despachado al cliente
`delivered`	**Entregado**	Pedido entregado exitosamente
`cancelled`	**Cancelado**	Pedido cancelado

Diagrama

10.3 Cambiar Estado de un Pedido

1. Ubica el pedido en la tabla
2. En la columna **"Acciones"**, encontrarás un **selector desplegable**
3. Selecciona el nuevo estado deseado
4. El estado se actualiza **automáticamente** (no necesitas presionar un botón de guardar)

> [!NOTE]
> El cambio de estado es inmediato y se refleja en la tabla al instante. El badge del estado cambiará de color automáticamente.

10.4 Ver Detalle de un Pedido

Para ver los detalles completos de un pedido:

1. Haz **clic en cualquier parte de la fila** del pedido (excepto el selector de estado)
2. Se expandirá una sección debajo de la fila con dos columnas:

Columna izquierda — Productos del Pedido:

Dato	Ejemplo
Nombre del producto	Vestido Floral Primavera
Talla	M
Color	Rosa
Cantidad	2
Subtotal	Q398.00

Columna derecha — Datos del Cliente:

Dato	Ejemplo
Email	cliente@ejemplo.com
Teléfono	5555-5555
Subtotal	Q398.00
Descuento	-Q50.00 (si aplica)
Total	Q348.00

> [!TIP]
> Haz clic nuevamente en la fila del pedido para **colapsar** los detalles.

---

11. Gestión de Promociones

La sección de promociones te permite crear y administrar códigos de descuento para tu tienda.

Ruta: `/admin/promos`

11.1 Lista de Códigos Promocionales

La vista principal muestra:

- **Encabezado**: Título "Códigos Promocionales" + contador (ej. "5 códigos creados")
- **Botón "➕ Nuevo Código"**: Para crear un nuevo código
- **Tabla** con todos los códigos existentes:

Columna	Descripción
Código	El código en mayúsculas (ej. WELCOME10)
Descuento	Valor y tipo (ej. "10%" o "Q50.00"). Muestra pedido mínimo si aplica
Usos	Formato `usados / máximo` (ej. "3 / 100" o "5 / ∞")
Estado	Badge "Activo" (verde) o "Inactivo" (rojo)
Acciones	Botones para activar/desactivar y eliminar

11.2 Crear un Nuevo Código Promocional

Haz clic en **"➕ Nuevo Código"** para mostrar el formulario de creación:

Campo	Tipo	Obligatorio	Descripción
Código	Texto	☑	Se convierte automáticamente a MAYÚSCULAS (ej. `VERANO20`)
Tipo	Desplegable	☑	**Porcentaje (%)** o **Monto Fijo (Q)**
Descuento	Número	☑	Valor del descuento (ej. 10 para 10% o 50 para Q50)
Pedido Mínimo (Q)	Número	—	Monto mínimo del pedido para poder usar el código
Límite de usos	Número	—	Número máximo de veces que se puede usar el código (vacío = ilimitado)

Paso a paso:

1. Haz clic en **"➕ Nuevo Código"**
2. Se mostrará un formulario debajo del encabezado
3. Ingresa el **código** (ej. `VERANO20` — se convertirá a mayúsculas automáticamente)
4. Selecciona el **tipo**: Porcentaje o Monto Fijo
5. Ingresa el **valor** del descuento
6. (Opcional) Configura un **pedido mínimo** en Quetzales
7. (Opcional) Establece un **límite de usos**
8. Presiona **"Crear Código"**

> [!TIP]
> Si dejas el campo "Límite de usos" vacío, el código podrá usarse un número **ilimitado** de veces. Es útil para promociones generales, pero para ofertas exclusivas recomendamos establecer un límite.

11.3 Activar / Desactivar un Código

Cada código tiene un botón toggle en la columna de acciones:

- Si está **activo**: Muestra **"⏸ Desactivar"** → clic para desactivar
- Si está **inactivo**: Muestra **"▶ Activar"** → clic para activar

> [!NOTE]
> Desactivar un código **no lo elimina**. Simplemente evita que los clientes lo usen hasta que lo reactives. Es útil para pausar promociones temporalmente.

11.4 Eliminar un Código

1. Ubica el código en la tabla
2. Haz clic en **"🗑 Eliminar"**
3. Aparecerá un **diálogo de confirmación**: *"¿Estás seguro de eliminar este código?"*
4. Confirma para eliminar o cancela para mantener el código

> [!WARNING]
> La eliminación de un código promocional es **permanente**. Si tienes pedidos que usaron este código, el historial del descuento se mantiene en los pedidos, pero el código ya no podrá ser utilizado.

🧭 12. Navegación del Panel Admin

12.1 Sidebar (Barra Lateral)

El panel de administración cuenta con una **barra lateral fija** en el lado izquierdo que contiene:



Sección	Contenido
Header	Logo "zzaly" + badge "Admin"
Navegación	📊 Dashboard · 📦 Productos · 🛒 Pedidos · 🏷 Promociones
Info del usuario	Avatar con inicial, nombre y email del admin
Footer	Botón "Cerrar Sesión" con ícono de puerta

> [!NOTE]
> El enlace activo se resalta visualmente para que siempre sepas en qué sección te encuentras.

12.2 Sidebar en Móvil

En pantallas pequeñas, el sidebar se comporta de manera diferente:

1. El sidebar se **oculta por defecto**
2. Aparece un botón **"☰"** (toggle) en la esquina superior
3. Al presionarlo, el sidebar se **desliza** sobre el contenido
4. Un **overlay oscuro** cubre el resto de la pantalla
5. Al hacer clic en el overlay o en cualquier enlace, el sidebar se cierra

---

PARTE 3 — PREGUNTAS FRECUENTES

❓ 13. Preguntas Frecuentes (FAQ)

👤 Para Clientes

<details>
<summary>🛒 ¿Cómo creo una cuenta en Zzaly?</summary>

Actualmente, Zzaly **no requiere crear una cuenta** para realizar compras. Utilizamos un sistema de **checkout como invitado**, lo que significa que puedes comprar de forma rápida sin necesidad de registrarte. Solo necesitas proporcionar tu información de contacto (email y teléfono) al momento de procesar tu pedido.

</details>

<details>
<summary>💳 ¿Qué métodos de pago aceptan?</summary>

La plataforma está preparada para integración con **Banco Industrial** como pasarela de pago, soportando la moneda local (Quetzales, GTQ). Actualmente el sistema procesa las órdenes y genera los identificadores de transacción necesarios. La activación completa del pago en línea se anunciará próximamente.

</details>

<details>
<summary>🚚 ¿Cuánto cuesta el envío?</summary>

¡El envío es **gratuito** en compras mayores a **Q500**! Puedes ver esta promoción permanente en la barra de anuncios superior del sitio.

</details>

<details>
<summary>🏷️ ¿Cómo aplico un código de descuento?</summary>

Para aplicar un código de descuento:

1. Agrega los productos que deseas al **carrito**
2. Ve a la página del **carrito** (`/carrito`)
3. Ubica la sección **"¿Tienes un código de descuento?"**
4. Escribe tu código en el campo de texto
5. Presiona el botón **"Aplicar"**
6. Si es válido, verás el descuento reflejado en el resumen del pedido

Los códigos pueden ser de **porcentaje** (ej. 10% de descuento) o de **monto fijo** (ej. Q50 de descuento).

</details>

<details>
<summary>💰 ¿En qué moneda están los precios?</summary>

Todos los precios en Zzaly están en **Quetzales guatemaltecos (Q)**, también conocidos como GTQ. Es la moneda oficial de Guatemala.

</details>

<details>
<summary>👗 ¿Cómo sé qué talla elegir?</summary>

Al seleccionar un producto, verás las tallas disponibles (XS, S, M, L, XL). Te recomendamos:

- Revisar la **guía de tallas** disponible en el pie de página del sitio
- Considerar tus medidas habituales
- Si tienes dudas, contacta a nuestro equipo en **hola@zzaly.com**

</details>

🔧 Para Administradores

<details>
<summary>🔑 ¿Cómo cambio mi contraseña de administrador?</summary>

Actualmente, el cambio de contraseña debe realizarse **directamente en la base de datos**. Para cambiar la contraseña:

1. Accede a tu panel de Supabase o a la base de datos PostgreSQL
2. Genera un nuevo hash de contraseña con **bcrypt**
3. Actualiza el campo `password_hash` en la tabla `zz_admins`

Se recomienda implementar una funcionalidad de cambio de contraseña en el panel de administración en futuras versiones.

</details>

<details>
<summary>⚠ ¿Qué significa "Stock Bajo" en el Dashboard?</summary>

La alerta de **"Stock Bajo"** indica el número de **variantes de producto** (combinaciones de talla y color) que tienen un stock **igual o menor a 5 unidades**.

Ejemplo: Si el "Vestido Floral" en talla M color Negro tiene solo 3 unidades, esa variante se contabiliza como una alerta de stock bajo.

¿Qué hacer?

1. Ve a la sección de **Productos** en el panel admin
2. Busca el producto con stock bajo
3. Haz clic en **"✏ Editar"**
4. En la sección de **Variantes**, actualiza el campo de **Stock** de la variante afectada
5. Guarda los cambios

</details>

<details>
<summary>📊 ¿Con qué frecuencia se actualizan las estadísticas del Dashboard?</summary>

Las estadísticas del Dashboard se cargan **en tiempo real** cada vez que accedes a la página. No hay caché, por lo que siempre verás los datos más actualizados.

</details>

<details>
<summary>🔄 ¿Mi sesión de admin expira?</summary>

Sí, la sesión de administrador tiene una duración de **24 horas**. Después de ese tiempo, el token JWT expira y serás redirigido automáticamente al formulario de login. Simplemente inicia sesión nuevamente para continuar.

</details>

<details>
<summary>📦 ¿Puedo crear un producto sin variantes?</summary>

No, cada producto requiere **al menos una variante** (combinación de talla, color y stock). Las variantes son necesarias para que los clientes puedan seleccionar opciones al comprar y para que el sistema gestione el inventario correctamente.

</details>

<details>
<summary>🏷 ¿Puedo crear un código promo con usos ilimitados?</summary>

Sí. Al crear un código promocional, simplemente deja el campo **"Límite de usos"** vacío. El código se mostrará con el símbolo ∞ (infinito) en la columna de usos y podrá ser utilizado sin restricciones de cantidad.

</details>

📐 Referencia Rápida de Rutas

Rutas del Cliente

Ruta	Página
`/`	Página de Inicio
`/nosotros`	Nosotros (Historia de Zzaly)
`/catalogo`	Catálogo de Productos
`/catalogo?category=vestidos`	Catálogo filtrado por Vestidos
`/catalogo?category=blusas`	Catálogo filtrado por Blusas
`/catalogo?category=pantalones`	Catálogo filtrado por Pantalones
`/catalogo?category=abrigos`	Catálogo filtrado por Abrigos
`/catalogo?category=accesorios`	Catálogo filtrado por Accesorios
`/carrito`	Carrito de Compras

Rutas del Administrador

Ruta	Página
`/admin/login`	Inicio de sesión del administrador
`/admin` o `/admin/dashboard`	Dashboard principal
`/admin/productos`	Gestión de Productos
`/admin/pedidos`	Gestión de Pedidos
`/admin/promos`	Gestión de Promociones

📞 Soporte y Contacto

Canal	Detalle
Email	hola@zzaly.com
Instagram	@zzaly
Pinterest	/zzaly
WhatsApp	Disponible en el pie de página

Hecho con ♥ por el equipo de Zzaly

© 2026 Zzaly. Todos los derechos reservados. Guatemala GT